



Recibe una cálida:

# ¡Bienvenida!

---

Te estábamos esperando 😊 +

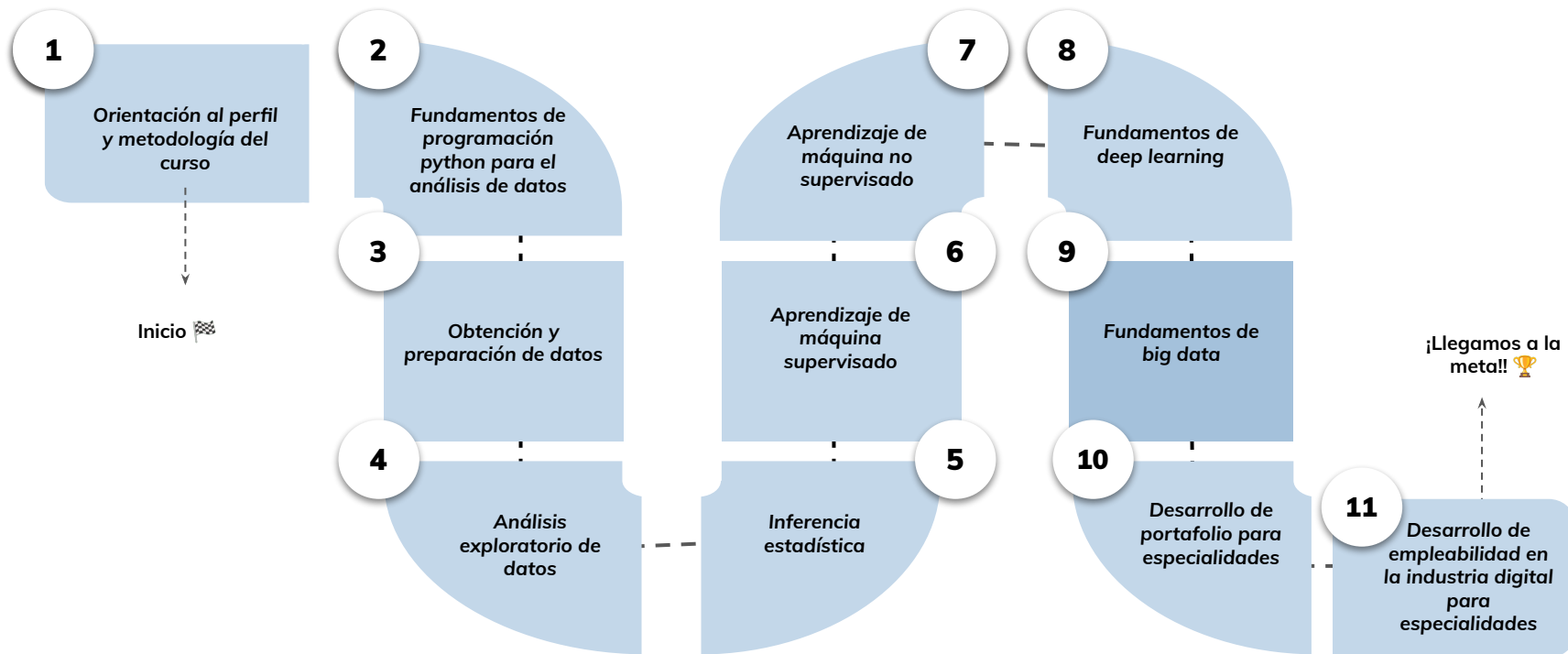


# › Procesamiento de datos: estructurados- Parte II

**Aprendizaje Esperado 4:** Manipular grandes volúmenes de datos utilizando spark sql para el procesamiento de grandes volúmenes de datos estructurados.

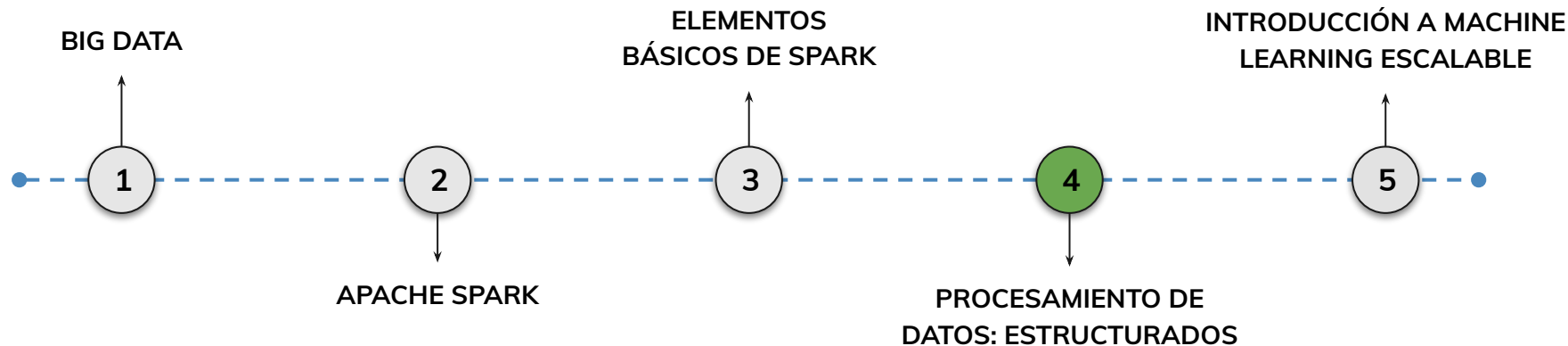
# Hoja de ruta

¿Cuáles skills conforman el programa? Fundamentos de Ciencia de Datos



# Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



# Learning Path

¿Cuáles temas trabajaremos hoy?

1.

## Consultas avanzadas y optimización en Spark SQL

Aprenderemos a aplicar consultas y transformaciones avanzadas sobre DataFrames en Spark SQL, integrando funciones de agregación, uniones y expresiones SQL optimizadas mediante Catalyst y Tungsten.

Consultas y transformaciones avanzadas

Optimización con Catalyst y Tungsten

Filtros y ordenamientos encadenados.

Agregaciones con funciones estadísticas.

Uso de selectExpr para expresiones SQL directas.

Visualización y análisis del plan de ejecución optimizado.



# Objetivos de aprendizaje

¿Qué aprenderás?



- Aplicar consultas y transformaciones avanzadas sobre DataFrames en Spark SQL.
- Utilizar funciones de agregación y unir múltiples DataFrames.
- Implementar expresiones SQL directamente en la API de DataFrame.
- Comprender el funcionamiento interno del Catalyst Optimizer y el proyecto Tungsten.
- Evaluar cómo estas optimizaciones impactan en el rendimiento de las consultas.

# Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Comprendimos qué es Spark SQL y sus ventajas sobre trabajar solo con RDD.
- Aprendimos a crear DataFrames desde diferentes fuentes (colecciones locales, CSV, JSON, Parquet, bases de datos vía JDBC).
- Ejecutamos consultas SQL sobre DataFrames y vistas temporales.
- Exploramos métodos para visualizar y analizar datos (`show()`, `printSchema()`, `describe()`, `summary()`).

# Procesamiento de datos: estructurados



# Métodos de creación de un DataFrame

La creación de un **DataFrame** en Apache Spark es un proceso flexible que puede realizarse a partir de diferentes fuentes de datos y estructuras existentes.

Spark proporciona diversos métodos para crear DataFrames, adaptándose a distintos escenarios y necesidades de análisis.

## Spark – Different ways to create DataFrame

- Create DataFrame from RDD
  - `rdd.toDF()`
  - `spark.createDataFrame(rdd)`
- Create Directly from Data (Seq, List)
  - `data.toDF()`
  - `spark.createDataFrame(rdd)`
- Create from File Data sources
  - CSV, Parquet, ORC, Avro, JSON, Text files
- Create from Databases
  - Hive, HBase, MySQL, Oracle
- Create DataFrame from existing DataFrame or Dataset



# Creación desde colecciones locales

Una de las formas más sencillas de crear un DataFrame es a partir de una colección de datos local, como una lista o un diccionario en Python.

```
# Crear una lista de tuplasdata = [("Juan", 30), ("Ana", 25), ("Carlos", 35)]# Crear un DataFrame con nombres de columnasdf = spark.createDataFrame(data, ["nombre", "edad"])
```

# Creación desde archivos CSV

Otra manera común de crear un DataFrame es mediante la lectura de archivos estructurados. Spark SQL soporta

múltiples formatos de archivo, como **CSV, JSON, Parquet, ORC y Avro**.  
Por ejemplo, para leer un archivo CSV:

```
df_csv = spark.read.csv("ruta/datos.csv", header=True, inferSchema=True)  
df_csv.show()
```

# Creación desde archivos JSON

También es posible crear un DataFrame leyendo archivos en formato **JSON**:

```
# Leer un archivo JSONdf_json =  
spark.read.json("ruta/a1/archivo.json")
```

```
import org.apache.spark.{SparkConf, SparkContext}  
import org.apache.spark.sql.{SparkSession, DataFrame}  
import org.apache.spark.sql.functions._  
  
object ConvertDFToJson extends App {  
  
  //Creating SparkSession  
  lazy val conf = new SparkConf().setAppName("ddv-parquet-generator").set("spark.default.parallelism", "2")  
    .setIfMissing("spark.master", "local[*]")  
  lazy val sparkSession = SparkSession.builder().config(conf).getOrCreate()  
  lazy val sparkContext: SparkContext = sparkSession.sparkContext  
  import sparkSession.implicits._  
  
  //Creating Input DataFrame  
  val inputDF = Seq(("Anamika",30,"Delhi",Map("First" -> "ABC")), ("Ananya",25,"Delhi",Map("Second" -> "XYZ")), ("Pritam",12,"U.P.",Map("Third" -> "MMO"))  
    .toDF( colNames = "Name", "Age", "City", "Data")  
  inputDF.show( truncate = false)  
  
  //For the last column, To change the key as City not Data :  
  //Reshaping the Columns using pivot  
  val transformedDF = inputDF.groupBy( col => "Name", cols => "Age", "City").pivot( pivotColumn => "city").agg(first( colName => "Data"))  
  transformedDF.show( truncate = false)  
  
  //Now make it by using toJson and collect it.  
  //Converting into json as expected output and collect it to print.  
  val jsonArray = transformedDF.toJSON.collect  
  jsonArray.foreach(println)  
}
```

# Creación desde archivos Parquet

El formato **Parquet**, que ofrece almacenamiento eficiente y optimizado para consultas analíticas, también puede ser utilizado para crear un DataFrame:

```
df_parquet = spark.read.parquet("ruta/datos.parquet")  
df_parquet.show()
```

# Creación desde bases de datos

Otra opción para crear un DataFrame es utilizando fuentes de datos externas, como bases de datos relacionales. Spark SQL permite conectarse a bases de datos mediante el protocolo **JDBC**.

```
df_jdbc = spark.read \  
    .format("jdbc") \  
    .option("url", "jdbc:mysql://localhost:3306/basededatos") \  
    .option("dbtable", "tabla") \  
    .option("user", "usuario") \  
    .option("password", "contraseña") \  
    .load()
```

# Creación desde consultas SQL

## Consultas sobre tablas existentes

Spark SQL permite crear DataFrames a partir de consultas SQL sobre otras tablas o DataFrames existentes:

```
# Registrar un DataFrame como vista temporaldf.createOrReplaceTempView("personas")# Crear un nuevo DataFrame a partir de una consulta SQLdf_mayores = spark.sql("SELECT * FROM personas WHERE edad > 30")
```

# Métodos de exploración de un DataFrame

Una vez creado un **DataFrame** en Apache Spark, es fundamental poder explorarlo para conocer su estructura, el tipo de datos que contiene y realizar un análisis preliminar antes de aplicar transformaciones o consultas avanzadas. Spark SQL ofrece múltiples métodos para explorar y examinar los DataFrames de manera eficiente.



# Visualización de datos con show()

## Método show()

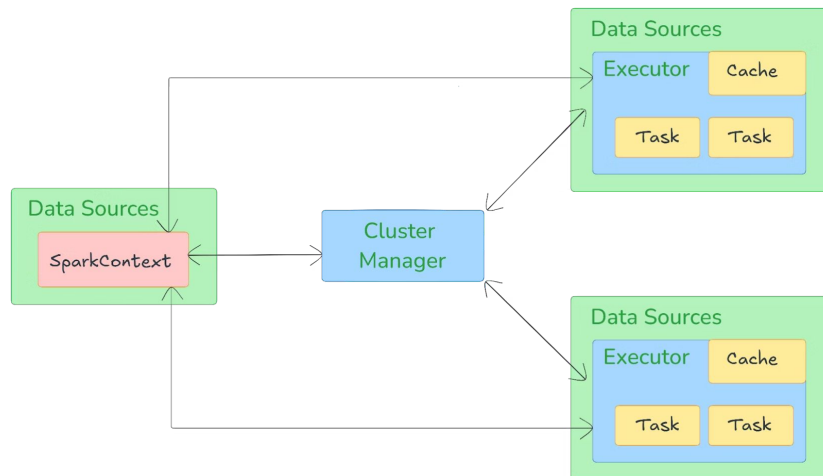
Uno de los métodos más utilizados es **show()**, que permite visualizar las primeras filas del DataFrame en la consola. Por defecto, muestra las primeras 20 filas, pero es posible especificar un número distinto:

```
# Mostrar las primeras 20 filasdf.show()# Mostrar las primeras 5 filasdf.show(5)# Mostrar todas las filas sin  
truncar el contenido df.show(truncate=False)
```

# Estructura del DataFrame

Otra función importante es **printSchema()**, que muestra la estructura del DataFrame, incluyendo los nombres de las columnas y los tipos de datos asociados. Esta función es útil para verificar que los datos han sido leídos correctamente y que los tipos son los esperados:

```
# Mostrar el esquema del DataFrame df.printSchema()
```



# Nombres de columnas

## Método `columns`

El método `columns` permite obtener una lista con los nombres de las columnas del DataFrame:

```
# Obtener los nombres de las columnas  
columnas = df.columns
```

# Resumen estadístico

Para obtener un resumen estadístico de las columnas numéricas, Spark SQL ofrece el método **describe()**. Este método calcula métricas como el conteo, la media, la desviación estándar, el valor mínimo y máximo:

```
# Obtener estadísticas descriptivasdf.describe().show()
```

# Información detallada

## Método `summary()`

Si se desea obtener información más detallada sobre cada columna, como la cantidad de valores nulos, los tipos de datos y las estadísticas básicas, se puede utilizar el método **`summary()`**:

```
# Obtener un resumen completo df.summary().show()
```

# Conteo de registros

El método **count()** permite conocer la cantidad total de registros que contiene el DataFrame:

```
# Contar el número de filasnum_filas = df.count()print(f"El DataFrame contiene {num_filas} filas")
```

# Primeras filas

Otra función útil es **head()**, que devuelve la primera fila del DataFrame como un objeto Row. También se puede utilizar **first()**, que cumple la misma función:

```
primera_fila = df.head()
print(primera_fila)
```

# Valores únicos

## Método **distinct()**

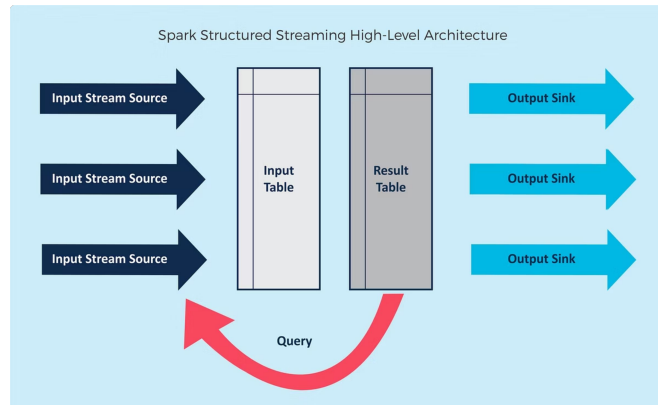
Para obtener los valores únicos de una columna específica, se puede utilizar el método **distinct()** combinado con **select()**:

```
# Obtener valores únicos de una columna df.select("departamento").distinct().show()
```



# Uso de DataFrames para consultas

Uno de los mayores beneficios de los **DataFrames** en Apache Spark es la posibilidad de realizar consultas sobre los datos de forma eficiente y utilizando un lenguaje familiar como **SQL** o funciones de la API de DataFrame. Esto facilita la exploración, transformación y análisis de grandes volúmenes de datos estructurados.



# Selección de columnas

## Método select()

Una de las formas más directas de consultar un DataFrame es utilizando métodos de filtrado y selección de columnas.

Por ejemplo, para seleccionar una columna específica, se puede utilizar el método **select()**:

```
# Seleccionar una columna
df.select("nombre").show()
# Seleccionar múltiples columnas
df.select("nombre",
"edad", "salario").show()
```

# Transformaciones encadenadas

También es posible aplicar múltiples transformaciones encadenadas. Por ejemplo, para seleccionar registros donde la edad sea mayor a 30 y ordenar los resultados por nombre:

```
# Filtrar y ordenar df.filter(df.edad >
30).orderBy("nombre").show()
```

| Transformations:                          |                                                                                                  |
|-------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>map(f : T -&gt; U):</code>          | <code>RDD&lt;T&gt; -&gt; RDD&lt;U&gt;</code>                                                     |
| <code>flatMap(f: T -&gt; Seq(U)):</code>  | <code>RDD&lt;T&gt; -&gt; RDD&lt;U&gt;</code>                                                     |
| <code>filter(f:T-&gt;Bool):</code>        | <code>RDD&lt;T&gt; -&gt; RDD&lt;T&gt;</code>                                                     |
| <code>groupByKey():</code>                | <code>RDD&lt;(K,V)&gt; -&gt; RDD&lt;(K,Seq[V])&gt;</code>                                        |
| <code>reduceByKey(F:(V,V)-&gt; V):</code> | <code>RDD&lt;(K,V)&gt; -&gt; RDD&lt;(K,V)&gt;</code>                                             |
| <code>union():</code>                     | <code>(RDD&lt;T&gt;,RDD&lt;T&gt;) -&gt; RDD&lt;T&gt;</code>                                      |
| <code>join():</code>                      | <code>(RDD&lt;(K,V)&gt;,RDD&lt;(K,W)&gt;) -&gt; RDD&lt;(K,(V,W))&gt;</code>                      |
| <code>cogroup():</code>                   | <code>(RDD&lt;(K,V)&gt;,RDD&lt;(K,W)&gt;)-&gt; RDD&lt;(K,(Seq&lt;V&gt;,Seq&lt;W&gt;))&gt;</code> |
| <code>crossProduct():</code>              | <code>(RDD&lt;T&gt;,RDD&lt;U&gt;) -&gt; RDD&lt;(T,U)&gt;</code>                                  |

| Actions:                            |                                              |
|-------------------------------------|----------------------------------------------|
| <code>count():</code>               | <code>RDD&lt;T&gt; -&gt; Long</code>         |
| <code>collect():</code>             | <code>RDD&lt;T&gt; -&gt; Seq&lt;T&gt;</code> |
| <code>reduce(f:(T,T)-&gt;T):</code> | <code>RDD&lt;T&gt; -&gt; T</code>            |
| <code>save(path:String):</code>     | Outputs RDD to a storage system e.g., HDFS   |

# Vistas temporales

Otra funcionalidad poderosa es la posibilidad de registrar un DataFrame como una **vista temporal (Temporary View)** y ejecutar consultas SQL estándar sobre él. Esto se hace con el método **createOrReplaceTempView()**:

```
# Crear una vista temporaldf.createOrReplaceTempView("empleados")
```

# Consultas SQL

Una vez registrada la vista, es posible ejecutar consultas SQL utilizando el método **spark.sql()**:

```
resultado = spark.sql("SELECT Nombre, Edad FROM empleados WHERE Edad > 30")  
resultado.show()
```

# Agregaciones

## Funciones de agregación

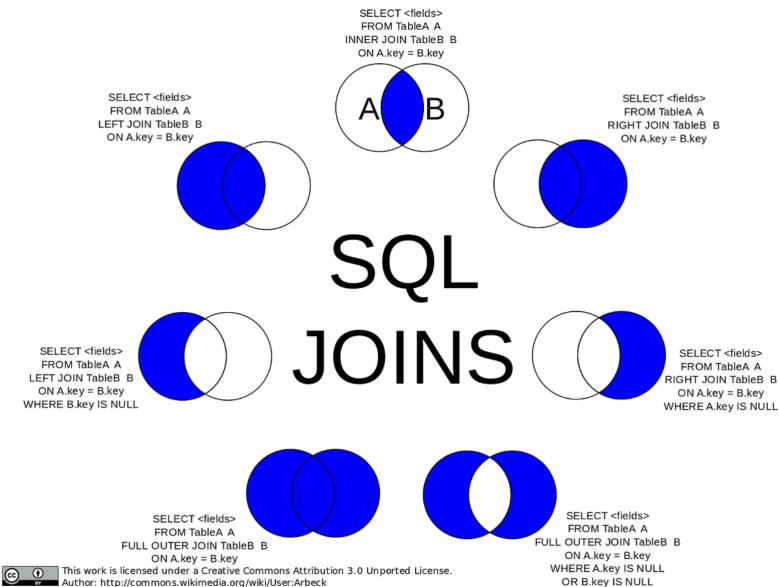
Además de consultas simples, se pueden realizar operaciones más complejas como agregaciones, uniones y combinaciones de DataFrames. Por ejemplo, para calcular la edad promedio de los empleados:

```
# Calcular la edad promediofrom pyspark.sql.functions import  
avgdf.select(avg("edad").alias("edad_promedio")).show()# Usando SQLspark.sql("SELECT AVG(edad) AS  
edad_promedio FROM empleados").show()
```

# Uniones de DataFrames

También es posible unir dos DataFrames utilizando el método **join()**, especificando la columna clave para la unión:

```
# Unir dos  
DataFramesdf_empleados.join(df_departamentos,  
df_empleados.id_dept == df_departamentos.id,  
"inner").show()
```



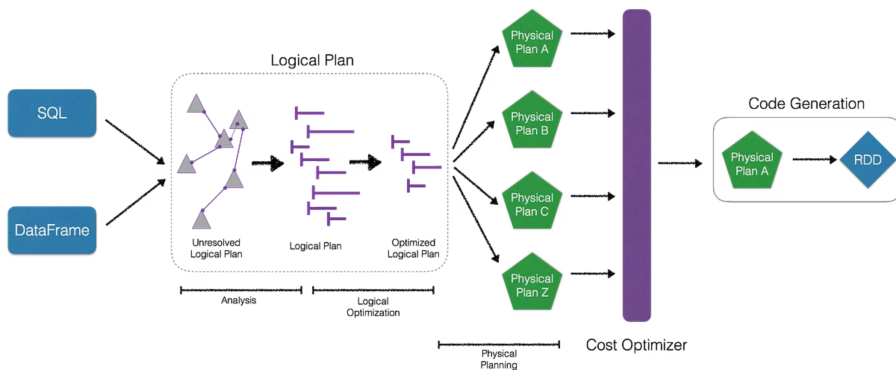
# Expresiones SQL

## Método selectExpr()

Otra operación común es el uso de expresiones SQL dentro de las funciones de DataFrame mediante **selectExpr()**, que permite aplicar expresiones SQL sin necesidad de registrar una vista:

```
# Usar expresiones SQL en selectExprdf.selectExpr( "nombre", "edad", "salario * 1.1 AS nuevo_salario", "CASE  
WHEN edad > 40 THEN 'Senior' ELSE 'Junior' END AS categoría").show()
```





# Optimización de consultas

Uno de los elementos que diferencian a **Apache Spark** de otras plataformas de procesamiento de datos es su capacidad para optimizar las consultas de forma automática y eficiente. Spark SQL incorpora dos proyectos clave para este propósito: **Catalyst** y **Tungsten**.

# Catalyst Optimizer



El **Catalyst Optimizer** es el motor de optimización de consultas de Spark SQL. Su objetivo principal es transformar las consultas lógicas realizadas por el usuario en planes físicos optimizados que puedan ejecutarse de manera eficiente en el clúster.

# Etapas de optimización de Catalyst

## Análisis lógico

Se valida la estructura de la consulta, se resuelven las referencias a columnas y se aplican reglas de simplificación.

## Optimización lógica

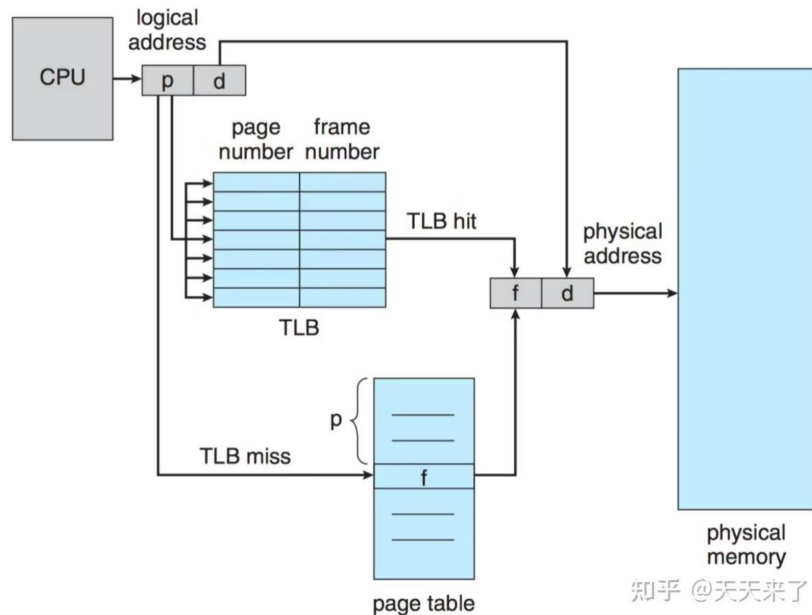
Se aplican reglas como la eliminación de columnas no utilizadas, el reordenamiento de filtros y la combinación de proyecciones.

## Generación de plan físico

Se define cómo se ejecutará la consulta en el clúster, incluyendo detalles sobre las operaciones que se ejecutarán en cada nodo.

# Proyecto Tungsten

Además de Catalyst, Spark SQL incorpora el proyecto **Tungsten**, que se centra en la optimización del uso de recursos a nivel bajo. Tungsten mejora la eficiencia de ejecución mediante técnicas como la gestión eficiente de la memoria, la generación de código bytecode y la ejecución vectorizada.



# Beneficios de Tungsten



## Optimización de memoria

Implementa una gestión de memoria fuera de la JVM (off-heap memory), reduciendo la latencia y mejorando la estabilidad.



## Generación de código

Genera código específico para cada consulta, eliminando sobrecargas y mejorando el rendimiento de ejecución.



## Ejecución vectorizada

Procesa múltiples registros en una sola operación, aprovechando las capacidades de los procesadores modernos.

# Transparencia para el usuario

Estas optimizaciones son transparentes para el usuario. No es necesario escribir código específico para beneficiarse de ellas, ya que se aplican automáticamente cuando se ejecutan consultas sobre DataFrames o vistas temporales.

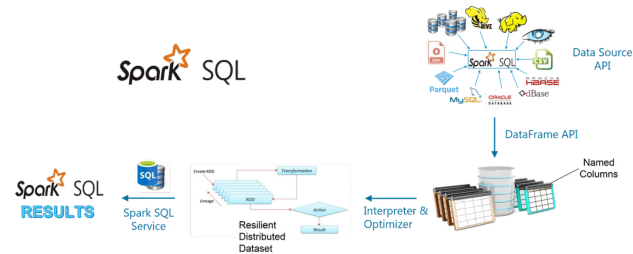


Figure: The flow diagram represents a Spark SQL process using all the four libraries in sequence

# Rendimiento comparativo

La combinación de Catalyst y Tungsten permite que Spark SQL ofrezca un rendimiento comparable e incluso superior al de sistemas tradicionales de bases de datos, mientras mantiene la flexibilidad del procesamiento distribuido.



# Conclusión

## **Procesamiento eficiente**

Catalyst y Tungsten son dos pilares fundamentales que permiten a Spark SQL ejecutar consultas de manera eficiente y escalable, aprovechando al máximo los recursos del clúster y mejorando la experiencia del usuario.

## **Base sólida**

El conocimiento adquirido en este manual proporciona una base sólida para que los estudiantes y profesionales puedan implementar soluciones de análisis de datos estructurados utilizando Apache Spark, integrando datos de diversas fuentes y aplicando técnicas avanzadas de consulta y transformación.





# Live Coding

## ¿En qué consistirá la Demo?

Exploraremos consultas avanzadas sobre grandes volúmenes de datos estructurados y veremos cómo Spark las optimiza automáticamente.

1. Filtrado y ordenamiento encadenado sobre un DataFrame.
2. Agregaciones y uso de funciones como `avg()`, `count()`, `sum()`.
3. Uniones internas y externas (`inner join`, `left join`).
4. Uso de `selectExpr` para aplicar expresiones SQL directamente.
5. Visualización del plan de ejecución y optimización automática con Catalyst.



Momento:

# Time-out!



5 -10 min.





Ejercicio

# Consultas optimizadas en Spark SQL

# Consultas optimizadas en Spark SQL

## Contexto: 🙌

Aplicarás técnicas avanzadas de consulta y optimización en Spark SQL para procesar datos estructurados de forma eficiente.

## Consigna: 📝

1. Carga un dataset en formato Parquet o CSV como DataFrame.
  - a. Realiza al menos tres consultas avanzadas que incluyan:
    - a. Filtros y ordenamientos encadenados.
    - b. Agregaciones con funciones estadísticas.
    - c. Una unión entre DataFrames.
    - d. Aplica expresiones SQL con selectExpr.
2. Observa el plan de ejecución (explain()) y analiza cómo Catalyst lo optimiza.

## Paso a paso: ⚙️

1. Inicia la sesión Spark y carga los datos.
2. Ejecuta las consultas y guarda los resultados.
3. Usa explain() para visualizar el plan lógico y físico.
4. Documenta cómo Catalyst y Tungsten intervienen en la optimización.

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





# Resumen

¿Qué logramos en esta clase?



- ✓ Ejecutamos consultas y transformaciones avanzadas con Spark SQL.
- ✓ Aplicamos agregaciones, uniones y expresiones SQL en DataFrames.
- ✓ Comprendimos el funcionamiento interno de Catalyst y Tungsten.
- ✓ Observamos cómo Spark optimiza automáticamente el plan de ejecución.
- ✓ Fortalecimos habilidades para procesar grandes volúmenes de datos estructurados de forma eficiente.



## ¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ———— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

